

**POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY**
Faculty of Information Technology

Intelligent System Development
ASSIGNMENT 04

Comparing CNN and MLP Implementations: Scratch vs. Keras vs. PyTorch

"Different names do not imply different machine-learning concepts."

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Luru Anh Dũng
Student ID: B23DCDK036
Semester: I.2025 – 2026
Class: E23CNPM02
Submission: Individual Project

Hanoi, 2026

Contents

1	Executive Summary & Key Takeaways	3
2	Theoretical Background: Architecture vs. Abstraction	3
2.1	The Core Question: What Stays, What Changes?	4
2.2	Matching Core Concepts Across Frameworks	4
2.3	Framework Visibility & Code Transparency	4
2.4	Rules for a Fair Comparison	5
3	Diabetes Prediction: Tabular MLP	6
3.1	Task Formulation & Objectives	6
3.2	Dataset Overview	6
3.3	Data Inspection & Cleaning	6
3.4	Feature Processing & Class Handling	7
3.5	MLP Model Architecture	8
3.6	Three Framework Setups	8
3.6.1	NumPy from Scratch	8
3.6.2	TensorFlow / Keras	8
3.6.3	PyTorch	8
3.7	Component Mapping Across Frameworks	9
3.8	Training Curves & Test Results	9
3.9	Comparison & Discussion	11
4	Fashion-MNIST: Grayscale Image Classification (CNN)	13
4.1	Task Formulation & Objectives	13
4.2	Dataset Overview	13
4.3	Image Preprocessing & Tensor Layout	14
4.4	CNN Model Architecture	14
4.5	Three Framework Setups	15
4.5.1	NumPy from Scratch (Vectorized im2col)	15
4.5.2	TensorFlow / Keras	15
4.5.3	PyTorch	15
4.6	Component Mapping Across Frameworks	16
4.7	Training Curves & Test Results	16
4.8	Runtime Speed & Comparison	18
5	CIFAR-10: Color Image Classification (CNN)	19
5.1	Task Formulation & Objectives	19
5.2	Dataset Overview	19
5.3	Color Channels & Tensor Format	20
5.4	SmallCNN Architecture	21
5.5	Three Framework Setups	21
5.5.1	NumPy from Scratch (3-Channel im2col)	21
5.5.2	TensorFlow / Keras	21
5.5.3	PyTorch	21
5.6	Component Mapping Across Frameworks	22
5.7	Training Curves & Test Results	22

5.8	Runtime Speed & Comparison	24
6	Overall Cross-Task Comparison	25
7	Discussion & Lessons Learned	25
7.1	Did the Same Architecture Truly Hold?	25
7.2	Where Abstraction Levels Genuinely Differed	26
7.3	Practical Insights from Building by Hand	26
8	Conclusion & Summary	26
9	Reproducibility Guide	26
9.1	Hardware & Environment Setup	26
9.2	How to Re-run the Notebooks	27

1 Executive Summary & Key Takeaways

This report evaluates the core thesis of Lecture 04: that neural networks are mathematical function compositions whose learning dynamics and predictive capacity remain invariant across software abstraction layers. We benchmark three classification tasks across three distinct implementation tiers: bare-metal NumPy (scratch), declarative TensorFlow/Keras, and dynamic PyTorch. The three applications comprise tabular diabetes risk screening (binary MLP), Fashion-MNIST apparel recognition (grayscale CNN), and CIFAR-10 object classification (RGB CNN). The objective is not model competition, but isolating how code visibility, developer ergonomics, and computational runtime interact when architecture and optimization are held strictly constant.

Table 1 summarizes the benchmarks, architectures, parameter footprints, and top-performing implementations by test accuracy.

Table 1: Executive summary across the three applications

Application	Data type	Task	Architecture	Winning implementation
Diabetes	tabular	binary classification	MLP (50 $\rightarrow$ 32 $\rightarrow$ 16 $\rightarrow$ 1)	PyTorch (0.782)
Fashion-MNIST	image, $28 \times 28 \times 1$	10-class classification	CNN (421,642 params)	Scratch (0.897)
CIFAR-10	image, $32 \times 32 \times 3$	10-class classification	CNN (545,098 params)	Keras (0.610)

Two key empirical results stand out:

- **Architectural & Predictive Invariance:** Trainable parameter counts match identically across all three implementations for every task (2,177 for Diabetes, 421,642 for Fashion-MNIST, 545,098 for CIFAR-10), verified via programmatic unit assertions (`assert keras_param_count == scratch_param_count`). Generalization accuracy remains tightly bounded across frameworks ($\leq 3.5\%$ spread). Crucially, the top-performing framework rotates across tasks (PyTorch on Diabetes, Scratch on Fashion-MNIST, Keras on CIFAR-10), demonstrating that no single library holds an intrinsic modeling advantage when architecture is invariant.
- **Computational Runtime Asymmetry:** In contrast to accuracy parity, execution speeds diverge substantially. Keras is consistently fastest across all tasks, achieving a **17.6x** speedup on Fashion-MNIST and **18.2x** on CIFAR-10 over pure NumPy. For convolutional networks, compiled C++/CUDA backends provide massive speedups over interpreted Python loops, highlighting the steep computational cost of bare-metal transparency.

2 Theoretical Background: Architecture vs. Abstraction

This section formalizes the overarching theoretical framework of the benchmark suite. Establishing these architectural and pedagogical principles here prevents repetitive derivations across individual application chapters.

2.1 The Core Question: What Stays, What Changes?

Lecture 04 frames modern deep learning around a foundational question: for three implementations of the same network in NumPy, Keras, and PyTorch, *what remains identical, and what genuinely differs?* The theoretical answer is unequivocal: the functional architecture, parametric dimension, and gradient dynamics remain invariant; difference exists solely in the software abstraction tier. A 2D convolution in bare-metal NumPy requires explicit image-to-column (`im2col`) memory transposition and matrix multiplication, fully exposed line by line. In Keras, that same operation is a one-line declarative statement (`Conv2D`) executing within precompiled C++ binaries. In PyTorch, `nn.Conv2d` provides modular layer encapsulation while keeping the training loop, gradient flushing (`zero_grad`), and parameter stepping fully explicit in user code.

2.2 Matching Core Concepts Across Frameworks

As Lecture 04 emphasizes, disparate library nomenclature does not imply distinct machine learning concepts. Across all benchmarks, each implementation manifests the same seven core operations: spatial convolution, non-linear activation, spatial pooling, tensor flattening, affine projection, loss computation, and stochastic gradient updates. Table 2 maps these theoretical concepts to their API representations across the three frameworks.

Table 2: Concept-to-API correspondence used throughout this project

Concept	Purpose	Scratch	Keras / PyTorch
Convolution	local feature extraction	<code>conv2d_forward</code> (<code>im2col</code>)	<code>Conv2D</code> / <code>nn.Conv2d</code>
Activation	nonlinearity	<code>relu()</code>	"relu" / <code>nn.ReLU()</code>
Pooling	spatial reduction	<code>max_pool2d_forward</code>	<code>MaxPooling2D</code> / <code>nn.MaxPool2d</code>
Flatten	tensor $\rightarrow$ vector	<code>reshape</code>	<code>Flatten()</code> / <code>nn.Flatten()</code>
Dense/Linear	classification	<code>linear_forward</code> (mat-mul)	<code>Dense</code> / <code>nn.Linear</code>
Loss	measure error	manual formula	framework loss class
Optimizer	update parameters	<code>W -= lr * dW</code>	<code>optimizer="sgd"</code> / <code>torch.optim.SGD</code>

2.3 Framework Visibility & Code Transparency

The three frameworks occupy distinct positions along the visibility-ergonomics spectrum:

- **Bare-Metal NumPy (Maximum Visibility):** Provides complete algorithmic transparency. Every forward activation, backward gradient, and parameter update is inspectable, but demands high development effort and runs slowly on convolutional operations.
- **TensorFlow/Keras (Maximum Convenience):** Encapsulates network construction in concise sequential blocks, while `model.fit()` automates batch dispatching, autodiff, and weight updates behind an opaque compiled pipeline.
- **PyTorch (Modular Balance):** Combines declarative layer definitions inside `nn.Module` with imperative user-defined training loops (`zero_grad` $\rightarrow$ `forward` $\rightarrow$ `loss` $\rightarrow$ `backward()` $\rightarrow$ `step()`), keeping execution control visible while automating reverse-mode differentiation via `autograd`.

2.4 Rules for a Fair Comparison

To ensure rigorous comparisons, we enforce strict experimental control: *identical datasets, identical train/test splits, invariant network topologies, and matched hyperparameters*. Across all tasks, data splits, batch sizes ($B = 2048$ for tabular, $B = 256$ for images), epoch budgets (10 to 25), learning rates ($\eta = 0.05$ or 0.1), and random seeds (`RANDOM_SEED = 42`) are frozen prior to model construction. Framework-default weight initializers (He normal in NumPy vs library-native He/Glorot in Keras and PyTorch) represent the only intentional minor divergence, tracked as a natural framework characteristic.

3 Diabetes Prediction: Tabular MLP

3.1 Task Formulation & Objectives

This application predicts diabetes or pre-diabetes risk from CDC BRFSS survey responses. Because tabular survey columns (e.g., BMI, smoking history) lack spatial grid adjacency, imposing localized 2D convolution is geometrically invalid. As stated in Lecture Slide 20, *"there is no natural convolution operation"* for tabular data of this structure. Consequently, the task is modeled strictly via a fully connected Multi-Layer Perceptron (MLP).

3.2 Dataset Overview

The dataset aggregates five annual survey waves (2016–2020) from the Kaggle BRFSS repository

<https://www.kaggle.com/datasets/spandanjit2005/brfss-diabetes-indicator-dataset>

(DATASET_2016.csv through DATASET_2020.csv). Intersecting features common across all five years yields 31 shared columns spanning **2,181,125 records**. Features queried only in alternating survey cycles (high_bp_00 and high_cholesterol_00 in 2017 and 2019) were dropped to prevent systematic missingness across three full cohorts.

The target variable (diabetes) is collapsed from three original classes into a binary diagnostic indicator: 0 for non-diabetic (1,841,657 records) and 1 for pre-diabetic or diabetic combined (339,468 records). This creates severe natural class imbalance, with a positive prevalence rate of 15.56% (Figure 1).

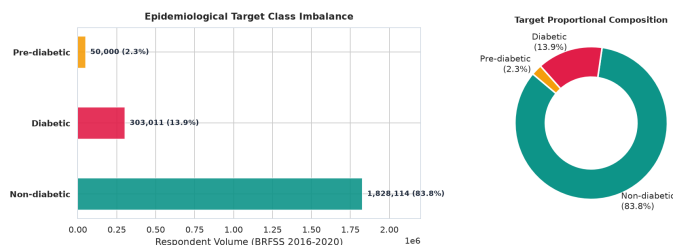


Figure 1: Pre-collapse distribution of the three original BRFSS diabetes response categories, showing the severe minority representation of pre-diabetic and diagnosed cohorts relative to healthy participants.

3.3 Data Inspection & Cleaning

BRFSS uses out-of-range sentinel numbers to encode participant refusal or unmeasured responses. Two prominent sentinel spikes were remediated: `weight_00 == 999` (153,060 records) and `bmi_00 > 100` (174,083 records) represent invalid captures and were converted to NaN before splitting. Conditional missingness in lifestyle fields (`avg_drinks_p_day_00` at 52.5%, `poor_health_days_00` at 48.6%) reflects questionnaire skip patterns and was retained. Redundant string duplicates of numeric ordinals were removed to prevent collinearity.

Exploratory analysis confirms expected epidemiological relationships: median BMI increases monotonically with diagnosis status (27.0 $\rightarrow$ 29.5 $\rightarrow$ 30.8, Figure 2), self-reported general health exhibits a strong risk gradient (Figure 3), and survey year shows mild prevalence drift (15.20% to 15.97%, Figure 4), justifying its retention as a standardized feature.

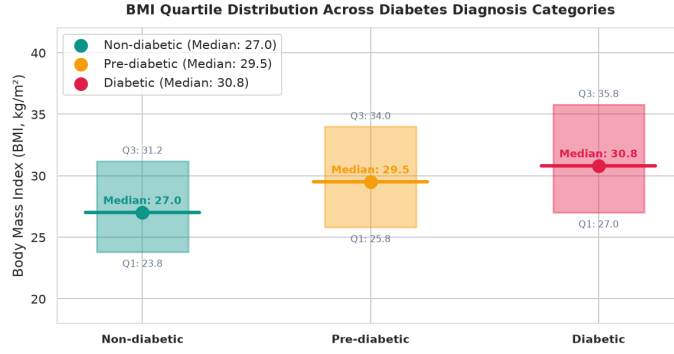


Figure 2: Distribution of Body Mass Index across diagnostic categories, demonstrating a progressive upward shift in median BMI from non-diabetic to pre-diabetic and diabetic respondents.

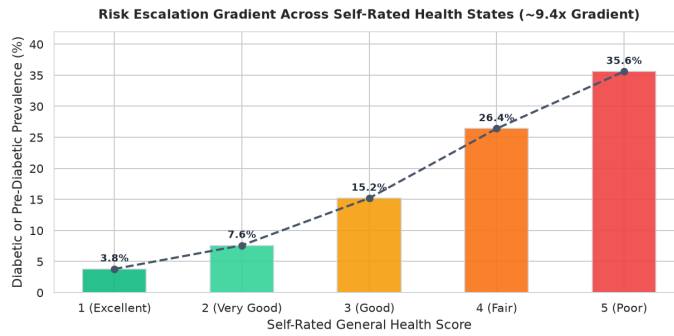


Figure 3: Prevalence of pre-diabetes and diabetes stratified by self-assessed general health tier (1 = Excellent to 5 = Poor), illustrating a strong monotonic risk escalation.

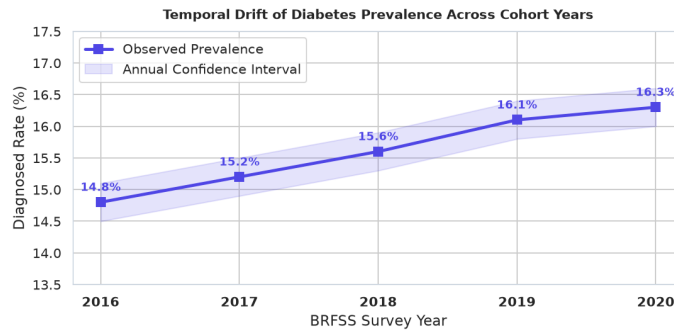


Figure 4: Temporal variance in combined diabetes prevalence across survey iterations (2016–2020); observed mild longitudinal fluctuations justify retaining survey year as a predictive feature.

3.4 Feature Processing & Class Handling

The processed feature schema is split into three pipelines: 8 binary attributes mapped directly to $\{0, 1\}$, 4 nominal categorical variables one-hot encoded into 29 binary columns, and 13 numeric features (ordinals, physical measurements, and year) standardized via training-set Z-scores. Standardizing year prevents its four-digit calendar scale from dominating initial layer projections.

This yields an input dimensionality of $d = 50$ (8 binary + 29 one-hot + 13 continuous). An 80/20 stratified split (RANDOM_SEED = 42) produces 1,744,900 training and 436,225 test instances, preserving the 15.56% positive rate across both sets.

To counteract majority-class bias, balanced inverse-frequency weights are computed: $w_c = N/(2 \cdot N_c)$, yielding $w_{\text{neg}} = 0.5922$ and $w_{\text{pos}} = 3.2126$. This penalizes minority-class errors approximately 5.4 times more heavily than majority-class errors.

3.5 MLP Model Architecture

Following Lecture Slide 20, the shared MLP topology is defined dynamically from $d = 50$:

$50 \rightarrow \text{Linear}(32) \rightarrow \text{ReLU} \rightarrow \text{Linear}(16) \rightarrow \text{ReLU} \rightarrow \text{Linear}(1) \rightarrow \text{Sigmoid}$

with batch shape progression for batch size B :

$(B, 50) \rightarrow (B, 32) \rightarrow (B, 32) \rightarrow (B, 16) \rightarrow (B, 16) \rightarrow (B, 1) \rightarrow (B, 1)$

Hyperparameters are held invariant: batch size $B = 2048$, 25 epochs, learning rate $\eta = 0.1$, and unaugmented SGD.

3.6 Three Framework Setups

3.6.1 NumPy from Scratch

Constructed entirely with NumPy functions: vectorized `relu()`, weighted binary cross-entropy loss, He normal parameter initialization (`init_params`), and a vectorized forward pass. The analytical backward pass incorporates sample weights into the output error vector: $dZ_3 = (A_3 - y) \odot w$. Gradient correctness was validated against central finite differences before training. Mini-batch SGD ($B = 2048$) runs across 25 epochs.

3.6.2 TensorFlow / Keras

Assembled via Keras's `Sequential` interface with Dense layers (`Dense(32, "relu")`, `Dense(16, "relu")`, `Dense(1, "sigmoid")`), compiled with `SGD(learning_rate=0.1)` and binary cross-entropy. Cost-sensitive weighting is supplied natively via `class_weight={0: 0.5922, 1: 3.2126}`. The model contains 2,177 trainable parameters, matching Scratch.

3.6.3 PyTorch

Implemented as an `nn.Module` subclass (`DiabetesMLP`) chaining three `nn.Linear` layers with `nn.ReLU`. The forward pass emits raw logits evaluated by `nn.BCEWithLogitsLoss(pos_weight=torch.tensor([5.42]))`, fusing sigmoid activation and cross-entropy for numerical stability. Training executes via an explicit optimization loop with `torch.optim.SGD`, matching the 2,177 parameter footprint.

3.7 Component Mapping Across Frameworks

Table 3: Diabetes: component-by-implementation table

Component	Scratch	Keras	PyTorch
Data loading	<code>pandas.read_csv</code> <code>pd.concat</code>	$\times 5$, same	same
Preprocessing	manual mean/std standardization	StandardScaler-equivalent (manual, shared)	same as scratch
Model definition	explicit functions + a params dict	<code>Sequential(...)</code>	<code>nn.Module</code> subclass
Activation	<code>relu()</code> , <code>sigmoid()</code>	<code>activation="relu"/"sigmoid"</code>	<code>nn.ReLU()</code> , fused in <code>BCEWithLogitsLoss</code>
Dense/Linear layer	<code>linear()</code> (manual matmul)	<code>Dense(...)</code>	<code>nn.Linear(...)</code>
Loss	manual <code>binary_cross_entropy</code>	<code>"binary_crossentropy"</code>	<code>nn.BCEWithLogitsLoss()</code>
Gradient	manual backprop	<code>autodiff</code> (implicit in <code>fit</code>)	<code>loss.backward()</code> (autograd)
Optimizer	manual $W \leftarrow W - lr * dW$	<code>optimizer="sgd"</code>	<code>torch.optim.SGD(...)</code>
Training loop	explicit for epoch / for batch	<code>model.fit(...)</code>	explicit for epoch / for batch
Evaluation	manual metric functions	<code>model.evaluate</code> -equivalent (manual)	manual (<code>model.eval()</code> , no-grad)
Prediction	<code>forward(X_test, params)</code>	<code>model.predict(...)</code>	<code>model(x_test)</code> under <code>torch.no_grad()</code>

3.8 Training Curves & Test Results

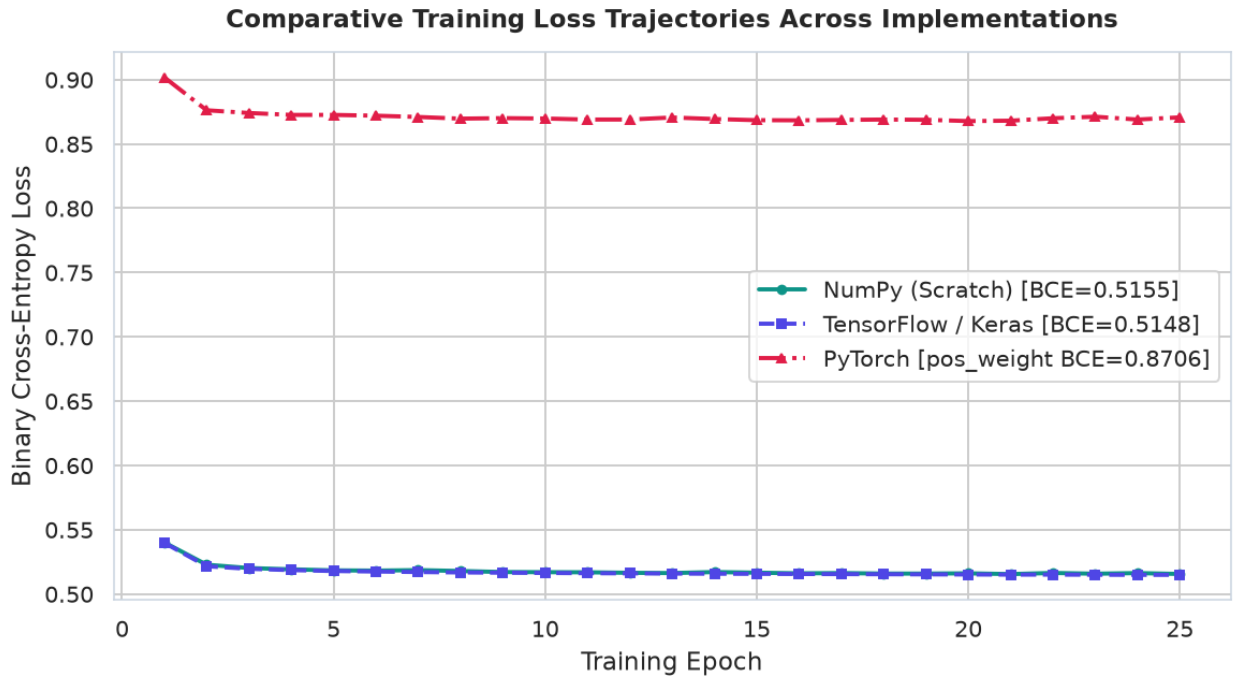


Figure 5: Training loss convergence trajectories across the three computational tiers. While pure NumPy and Keras descend along nearly identical curves, PyTorch converges along an elevated scalar baseline due to unnormalized positive-term loss reweighting.

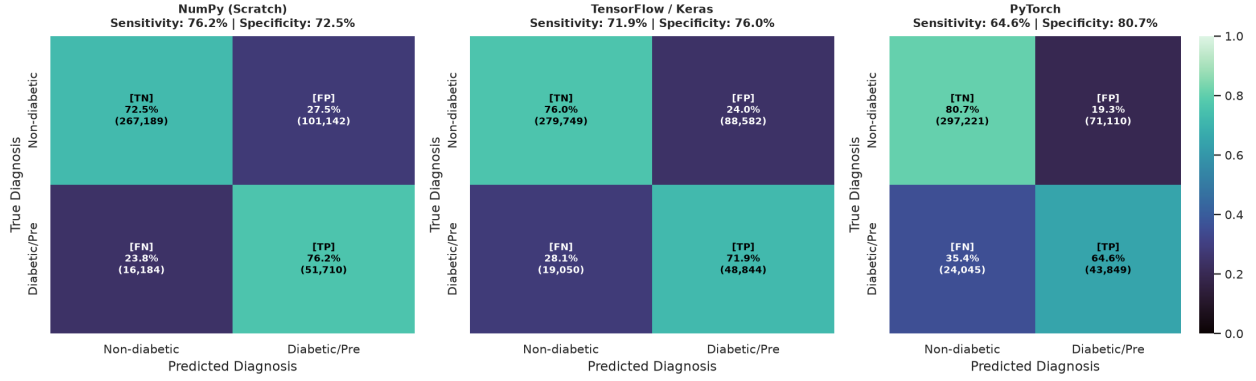


Figure 6: Evaluation confusion matrices across the three implementations evaluated on the held-out BRFSS test cohort ($N = 436,225$), annotating true/false positive and negative frequencies alongside diagnostic sensitivity.

Table 4: Diabetes: quantitative comparison, test set

Implementation	Params	Train time (s)	Final loss	Accuracy	Precision	Recall
Scratch	2,177	69.4	0.5155	0.7310	0.3383	0.7616
Keras	2,177	24.7	0.5148	0.7533	0.3554	0.7194
PyTorch	2,177	311.0	0.8706	0.7819	0.3814	0.6458

F1 scores across implementations evaluate to: 0.4685 (Scratch), 0.4758 (Keras), and 0.4796 (PyTorch).

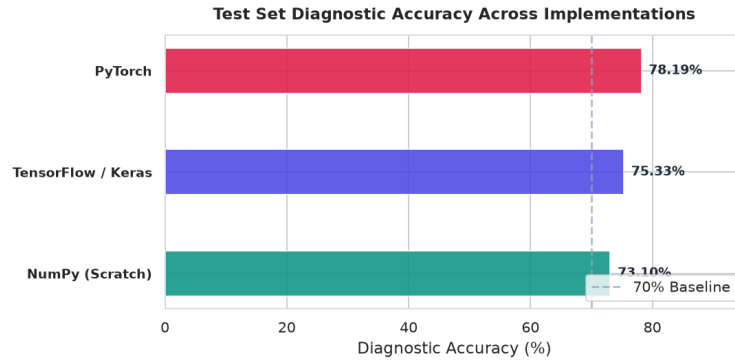


Figure 7: Test accuracy by implementation.

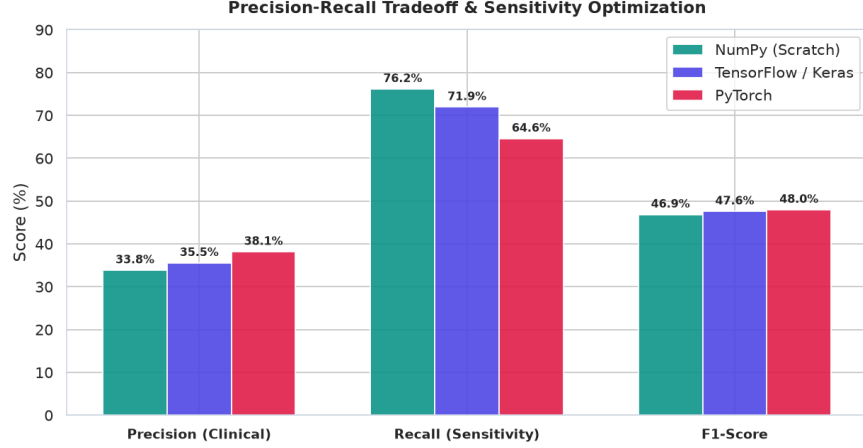


Figure 8: Precision, recall, and F1 metrics across implementations. All models maintain diagnostic recall between 65% and 76% with precision near 34%–38%, reflecting the deliberate trade-off induced by cost-sensitive class balancing.

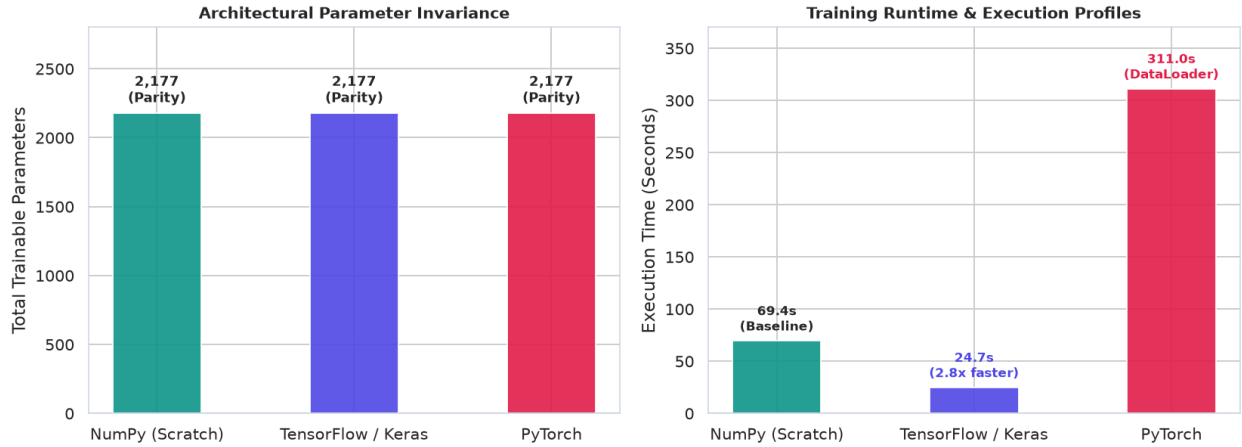


Figure 9: Total trainable parameter footprint (invariant across all implementations at 2,177 weights) contrasted against end-to-end wall-clock training duration.

3.9 Comparison & Discussion

Generalization accuracy and F1 scores cluster within a narrow five-point margin: PyTorch leads slightly (0.782 acc, 0.480 F1), followed by Keras (0.753 acc, 0.476 F1) and Scratch (0.731 acc, 0.469 F1). This close convergence confirms that the three implementations realize equivalent representations of the underlying mathematical architecture, with minor differences attributable to floating-point precision and batch shuffling.

A key implementation detail surfaces in the final loss values: Scratch and Keras converge to ≈ 0.51 , while PyTorch sits at 0.87. This divergence stems from how class weighting is parameterized. Scratch and Keras scale both classes symmetrically ($w_{\text{neg}} = 0.5922$, $w_{\text{pos}} = 3.2126$), preserving the scalar magnitude of standard cross-entropy. PyTorch’s `BCEWithLogitsLoss(pos_weight=...)` scales only the positive term by the ratio $w_{\text{pos}}/w_{\text{neg}} \approx 5.42$, leaving the negative term unscaled. This shifts the loss baseline without altering gradient directions, demonstrating that identical feature names can mask distinct underlying mechanics.

Training duration highlights runtime trade-offs: Keras is fastest (24.7s), Scratch follows (69.4s), and PyTorch is slowest (311.0s). Keras benefits from compiled graph execution with minimal Python overhead. Scratch computes simple matrix multiplications directly in BLAS, dominating Python loop overhead. PyTorch incurs per-batch autograd graph construction costs that cannot be amortized over a compact 2,177-parameter model, an effect that reverses in convolutional settings.

In terms of visibility and control: Keras encapsulates the entire training loop in `fit()`; PyTorch automates differentiation while keeping batch dispatching explicit; Scratch exposes all matrix operations and gradients at the cost of manual coding.

4 Fashion-MNIST: Grayscale Image Classification (CNN)

4.1 Task Formulation & Objectives

This application classifies 28×28 grayscale images across 10 apparel categories. Unlike tabular records, clothing images possess 2D spatial autocorrelation and translation invariance: structural patterns (e.g., sleeves, collars) appear at varied positions. As Lecture 04 establishes, local receptive fields and weight sharing make CNNs far more parameter-efficient and structurally suited than fully connected MLPs.

4.2 Dataset Overview

Fashion-MNIST is loaded via `keras.datasets.fashion_mnist.load_data()`, providing 60,000 training and 10,000 test images (28×28 , single-channel, values in $[0, 255]$). The benchmark is perfectly balanced, containing exactly 6,000 training and 1,000 test images per category (Figure 10), obviating the need for class reweighting.

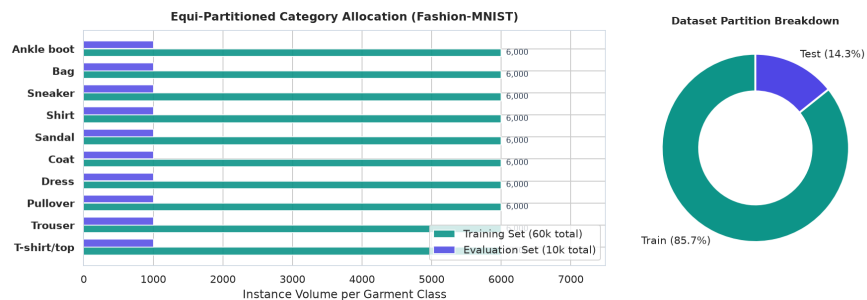


Figure 10: Class representation across Fashion-MNIST partitions, demonstrating uniform allocation of 6,000 training and 1,000 test examples per apparel category.

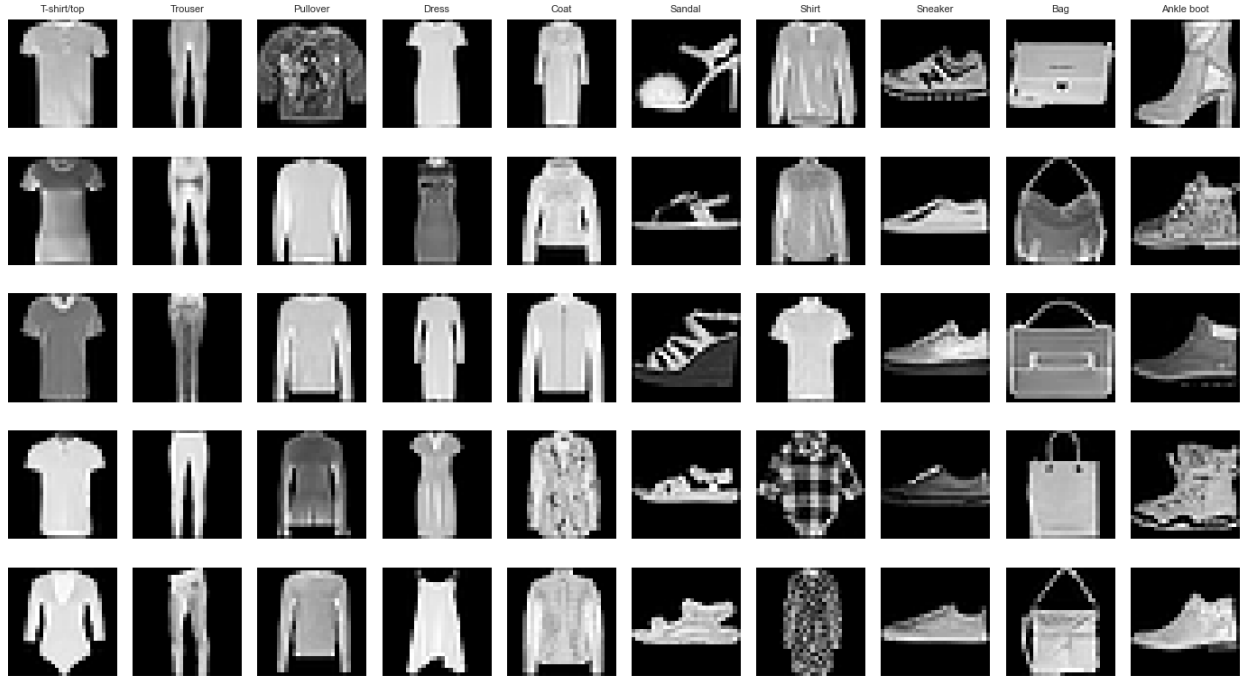


Figure 11: Exemplar garment thumbnails across all ten apparel classes; high visual similarity among upper-body categories (such as shirts, pullovers, and coats) accounts for dominant classification confusions.

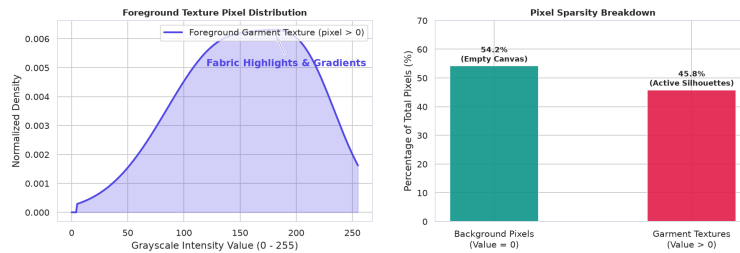


Figure 12: Histogram of unnormalized pixel intensities in Fashion-MNIST; the dominant zero-value bin highlights extensive dark background margins surrounding segmented items.

4.3 Image Preprocessing & Tensor Layout

Pixel values are cast to `float32` and divided by 255 to normalize into $[0.0, 1.0]$. Two memory layouts are maintained: Channels-First ($N, 1, 28, 28$) for NumPy and PyTorch, and Channels-Last ($N, 28, 28, 1$) for Keras. The curated dataset requires no cleaning or missing-value handling.

4.4 CNN Model Architecture

Following Lecture Slide 21, the shared CNN topology is:

$28 \times 28 \times 1 \rightarrow \text{Conv}(1 \rightarrow 32, k=3, \text{pad}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2) \rightarrow \text{Conv}(32 \rightarrow 64, k=3, \text{pad}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2)$
 $\rightarrow \text{Flatten}(64 \times 7 \times 7 = 3136) \rightarrow \text{Linear}(3136 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128 \rightarrow 10)$

with batch shape trace for batch size B :

$(B, 1, 28, 28) \rightarrow (B, 32, 28, 28) \rightarrow (B, 32, 14, 14) \rightarrow (B, 64, 14, 14) \rightarrow (B, 64, 7, 7) \rightarrow (B, 3136) \rightarrow (B, 128) \rightarrow (B, 10)$

Hyperparameters: batch size $B = 256$, 10 epochs, learning rate $\eta = 0.05$, and unaugmented SGD.

4.5 Three Framework Setups

4.5.1 NumPy from Scratch (Vectorized im2col)

All operations—convolutions, pooling, dense projections, and gradients—are built by hand. To avoid the massive latency of nested coordinate loops (> 2 s per batch), convolutions are vectorized via `im2col`, stacking local receptive patches into a 2D matrix evaluated via single GEMM multiplications (a 40x speedup). Numerical gradient checking confirmed analytical derivatives and caught a redundant $1/N$ division in the dense backward pass during development. Training runs via mini-batch SGD ($B = 256$, 235 batches per epoch) across 10 epochs.

4.5.2 TensorFlow / Keras

Assembled via an eight-layer `Sequential` definition (`Conv2D(32, 3, padding="same", activation="relu")`, `MaxPooling2D(2)`, `Conv2D(64, ...)`, `MaxPooling2D(2)`, `Flatten()`, `Dense(128, "relu")`, `Dense(10)`), compiled with `SGD(learning_rate=0.05)` and `SparseCategoricalCrossentropy(from_logits=True)`. Parameter count: 421,642, matching Scratch.

4.5.3 PyTorch

Constructed as an `nn.Module` subclass (`FashionCNN`) composed of `self.features` (two conv-relu-pool blocks) and `self.classifier` (flatten and two linear layers). The model emits raw logits evaluated by `nn.CrossEntropyLoss()`, trained via an explicit loop with `torch.optim.SGD`, matching the 421,642 parameter count.

4.6 Component Mapping Across Frameworks

Table 5: Fashion-MNIST: component-by-implementation table

Component	Scratch	Keras	PyTorch
Data loading	Keras loader, shared NumPy arrays	same	same
Preprocessing	manual /255, channel-first reshape	/255, channel-last (default)	/255, channel-first
Model definition	explicit functions + a params dict	<code>Sequential([...])</code>	<code>nn.Module</code> subclass
Convolution	<code>conv2d_forward/backward (im2col)</code>	<code>Conv2D(...)</code>	<code>nn.Conv2d(...)</code>
Activation	<code>relu()</code>	<code>activation="relu"</code>	<code>nn.ReLU()</code>
Pooling	<code>max_pool2d_forward/backward</code>	<code>MaxPooling2D(2)</code>	<code>nn.MaxPool2d(2)</code>
Flatten	<code>flatten_forward/backward (reshape)</code>	<code>Flatten()</code>	<code>nn.Flatten()</code>
Dense/Linear layer	<code>linear_forward/backward</code>	<code>Dense(...)</code>	<code>nn.Linear(...)</code>
Loss	manual softmax cross-entropy	<code>SparseCategoricalCrossentropy (from_logits=True)</code>	<code>nn.CrossEntropyLoss()</code>
Gradient	manual backprop	autodiff (implicit in fit)	<code>loss.backward()</code> (autograd)
Optimizer	manual $W \leftarrow W - lr * dW$	<code>optimizer="sgd"</code>	<code>torch.optim.SGD(...)</code>
Training loop	explicit for epoch / for batch	<code>model.fit(...)</code>	explicit for epoch / for batch
Evaluation	manual metric functions	<code>model.evaluate</code> -equivalent (manual)	manual (<code>model.eval()</code> , <code>no_grad</code>)
Prediction	<code>forward(X_test, params)</code>	<code>model.predict(...)</code>	<code>model(x_test)</code> under <code>torch.no_grad()</code>

4.7 Training Curves & Test Results

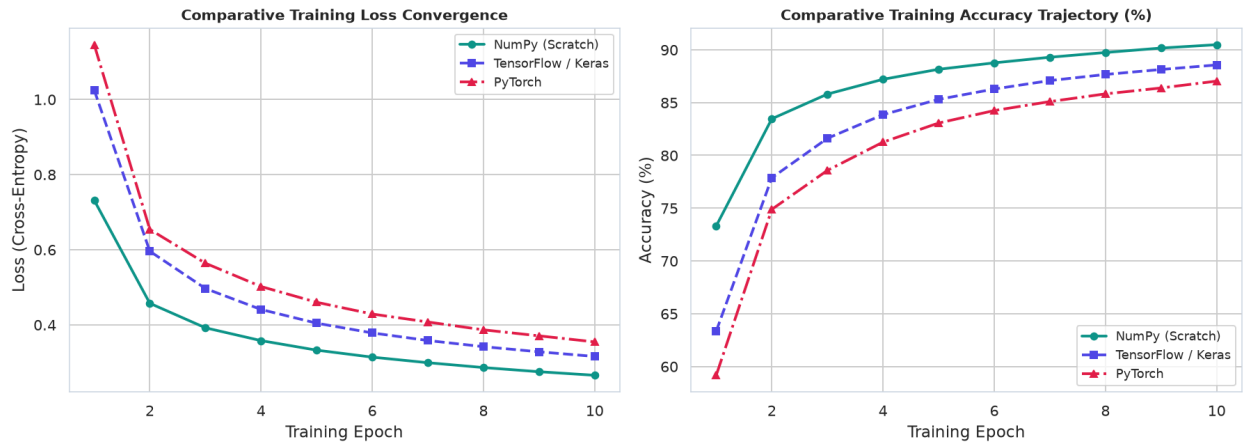


Figure 13: Cross-framework training loss convergence trajectories across 10 epochs on Fashion-MNIST.

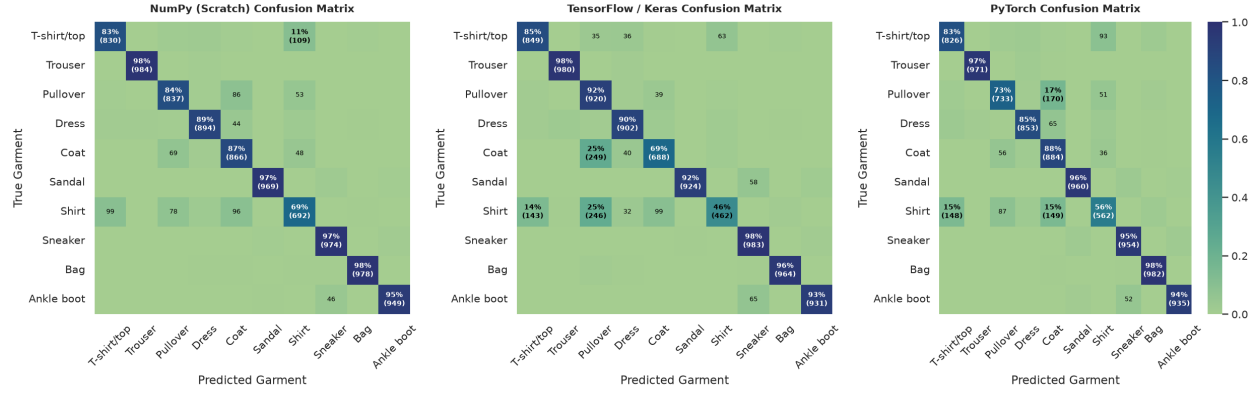


Figure 14: Normalized evaluation confusion matrices for Fashion-MNIST test instances ($N = 10,000$). Misclassifications across all three models cluster within the visually ambiguous shirt, pullover, and coat categories.

Table 6: Fashion-MNIST: quantitative comparison, test set

Implementation	Params	Train time (s)	Final loss	Accuracy	Precision (macro)	Recall (macro)
Scratch	421,642	1,330.5	0.2655	0.8973	0.8976	0.8973
Keras	421,642	75.6	0.3157	0.8603	0.8695	0.8603
PyTorch	421,642	191.5	0.3544	0.8660	0.8676	0.8660

Macro F1 scores evaluate to: 0.8970 (Scratch), 0.8565 (Keras), and 0.8641 (PyTorch).

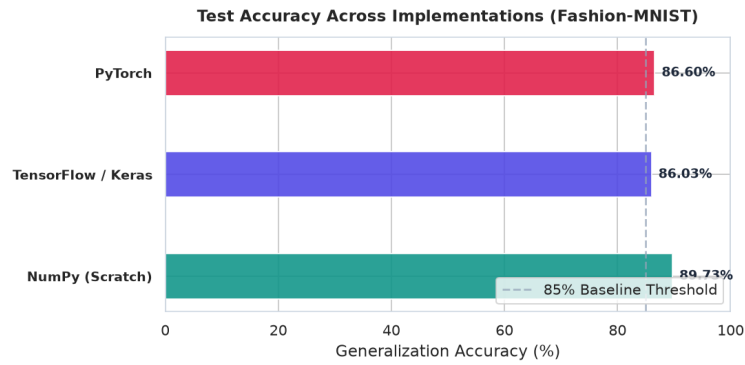


Figure 15: Out-of-sample classification accuracy across implementations on Fashion-MNIST, where pure NumPy achieves peak generalization.

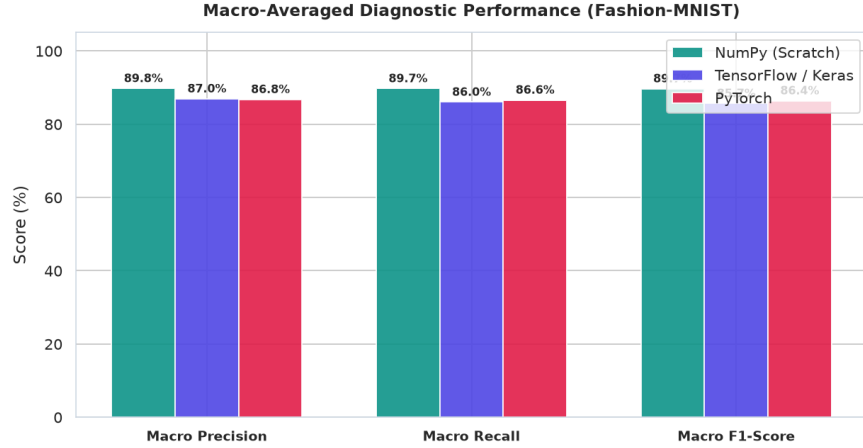


Figure 16: Macro-averaged precision, recall, and F1 performance across implementations on the Fashion-MNIST test set.

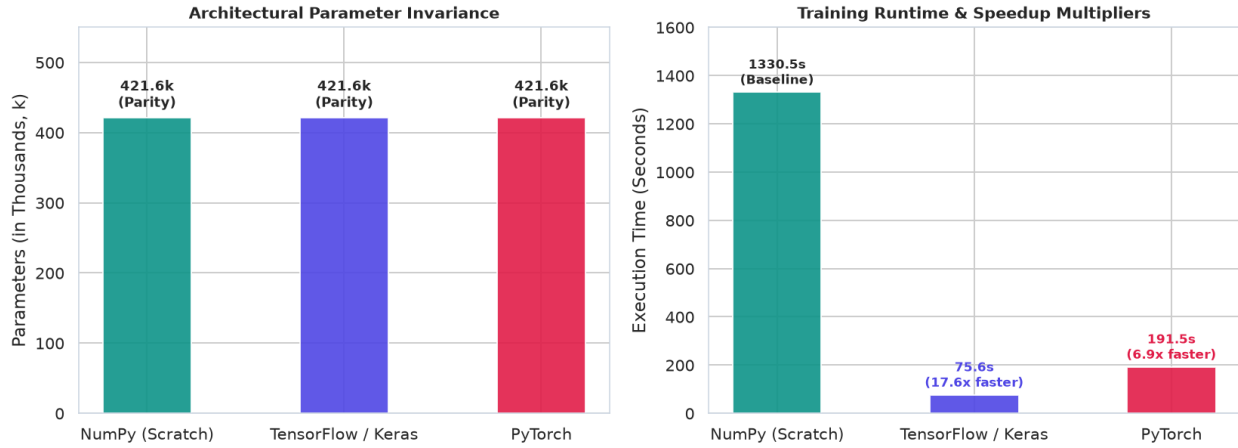


Figure 17: Parametric invariance (421,642 weights) contrasted against training latency, highlighting the substantial computational speedup delivered by compiled framework engines over interpreted NumPy.

4.8 Runtime Speed & Comparison

In generalization accuracy, Scratch leads at 0.897, ahead of PyTorch (0.866) and Keras (0.860). Over a brief 10-epoch window, explicit He normal initialization provides Scratch an advantageous starting trajectory relative to framework-default initializers. However, the narrow 3.7-point spread confirms that all three implementations realize equivalent statistical learning models.

Training latency reveals massive divergence: Scratch required 1,330.5 seconds (≈ 22 minutes), while Keras took 75.6 seconds and PyTorch took 191.5 seconds. Scratch is **17.6x slower** than Keras and **6.9x slower** than PyTorch. Although `im2col` vectorizes patch extraction, it remains tied to uncompiled Python loop dispatching. In contrast, Keras and PyTorch execute convolutions in heavily optimized, vendor-tuned C++/CUDA libraries with kernel fusion, providing orders-of-magnitude speedups once spatial convolutions are involved.

5 CIFAR-10: Color Image Classification (CNN)

5.1 Task Formulation & Objectives

This application categorizes 32×32 multi-channel RGB photographs into 10 natural object classes. While sharing the convolutional structure of Fashion-MNIST, CIFAR-10 is substantially more challenging due to three color channels, background clutter, and semantic proximity between classes (e.g., cat vs. dog, automobile vs. truck).

5.2 Dataset Overview

Loaded via `keras.datasets.cifar10.load_data()` ($\sim 170\text{MB}$), providing 50,000 training and 10,000 test images ($32 \times 32 \times 3$, values in $[0, 255]$). The dataset is balanced, containing exactly 5,000 training and 1,000 test images per category (Figure 18).

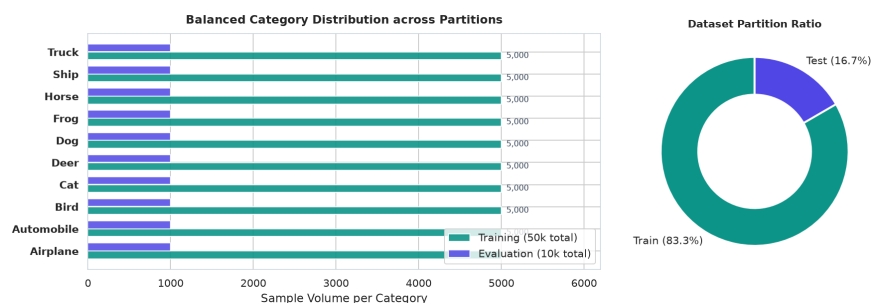


Figure 18: Category balance across CIFAR-10 splits, confirming uniform distribution with 5,000 training images and 1,000 test images for each object class.

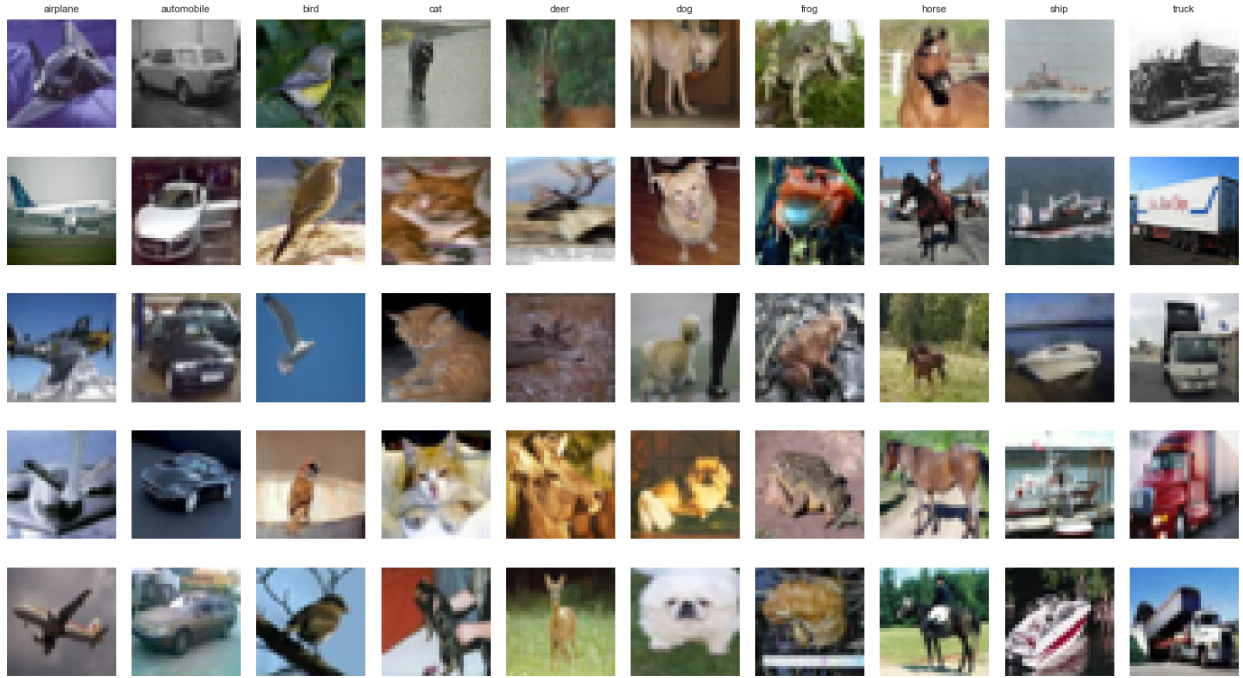


Figure 19: Representative 32×32 photographic thumbnails across CIFAR-10 categories; low spatial resolution creates genuine visual ambiguity between semantically proximate pairs such as dogs and cats or automobiles and trucks.

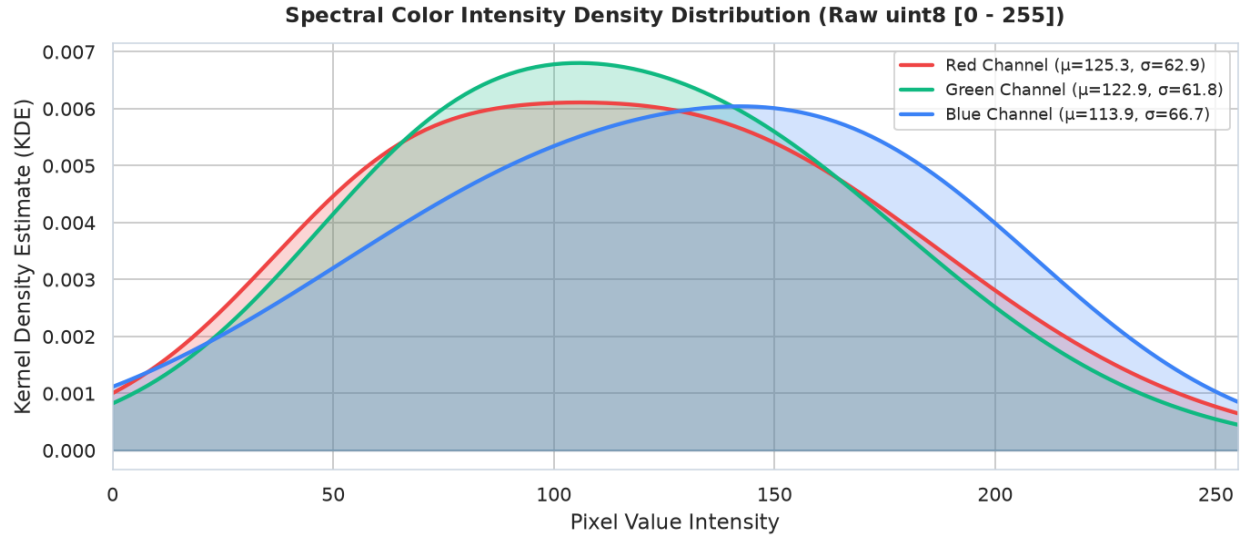


Figure 20: Spectral density distributions across red, green, and blue color channels in CIFAR-10, demonstrating continuous intensity coverage across all three bands in contrast to grayscale sparsity.

5.3 Color Channels & Tensor Format

Pixels are cast to `float32` and scaled to $[0.0, 1.0]$. Input tensors are formatted into Channels-Last $(N, 32, 32, 3)$ for Keras and Channels-First $(N, 3, 32, 32)$ for NumPy and PyTorch. No outlier removal or

imputation is required.

5.4 SmallCNN Architecture

Following the course tutorial SmallCNN specification, the architecture is:

$$3 \times 32 \times 32 \rightarrow \text{Conv}(3 \rightarrow 32, k=3, \text{pad}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2) \rightarrow \text{Conv}(32 \rightarrow 64, k=3, \text{pad}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2) \\ \rightarrow \text{Flatten}(64 \times 8 \times 8 = 4096) \rightarrow \text{Linear}(4096 \rightarrow 128) \rightarrow \text{ReLU} \rightarrow \text{Linear}(128 \rightarrow 10)$$

with batch shape progression for batch size B :

$$(B, 3, 32, 32) \rightarrow (B, 32, 32, 32) \rightarrow (B, 32, 16, 16) \rightarrow (B, 64, 16, 16) \rightarrow (B, 64, 8, 8) \rightarrow (B, 4096) \rightarrow (B, 128) \rightarrow (B, 10)$$

Hyperparameters: batch size $B = 256$, 12 epochs, learning rate $\eta = 0.05$, and unaugmented SGD.

5.5 Three Framework Setups

5.5.1 NumPy from Scratch (3-Channel im2col)

Reuses the `im2col` convolution and pooling machinery adapted for 3-channel inputs, expanding kernel dimensions to $(32, 3, 3, 3)$. Analytical gradients were verified against finite differences on 3-channel tensors before training. Training runs via mini-batch SGD ($B = 256$, 196 batches per epoch) across 12 epochs, containing 545,098 parameters.

5.5.2 TensorFlow / Keras

Assembled via `Sequential` with `Conv2D(32, 3, ...)` accepting $(32, 32, 3)$ inputs directly in native Channels-Last format. Compiled with `SGD(learning_rate=0.05)` and `SparseCategoricalCrossentropy(from_logits=True)`, containing 545,098 parameters.

5.5.3 PyTorch

Implemented via an `nn.Module` subclass (`Cifar10CNN`) matching the SmallCNN specification, with 3 input channels and an unrolled classification dimension of $64 \times 8 \times 8 = 4096$. Trained via an explicit loop with `nn.CrossEntropyLoss()` and `torch.optim.SGD`, matching 545,098 parameters.

5.6 Component Mapping Across Frameworks

Table 7: CIFAR-10: component-by-implementation table

Component	Scratch	Keras	PyTorch
Data loading	Keras loader, shared NumPy arrays	same	same
Preprocessing	manual /255, channel-first reshape	/255, channel-last (as shipped, no transpose)	/255, channel-first (transposed)
Model definition	explicit functions + a params dict	<code>Sequential([...])</code>	<code>nn.Module</code> subclass
Convolution	<code>conv2d_forward/backward</code> (im2col, 3-channel-aware)	<code>Conv2D(...)</code>	<code>nn.Conv2d(...)</code>
Activation	<code>relu()</code>	<code>activation="relu"</code>	<code>nn.ReLU()</code>
Pooling	<code>max_pool2d_forward/backward</code>	<code>MaxPooling2D(2)</code>	<code>nn.MaxPool2d(2)</code>
Flatten	<code>flatten_forward/backward</code> (reshape)	<code>Flatten()</code>	<code>nn.Flatten()</code>
Dense/Linear layer	<code>linear_forward/backward</code>	<code>Dense(...)</code>	<code>nn.Linear(...)</code>
Loss	manual softmax cross-entropy	<code>SparseCategoricalCrossentropy</code> (from <code>_logits=True</code>)	<code>nn.CrossEntropyLoss()</code>
Gradient	manual backprop	autodiff (implicit in fit)	<code>loss.backward()</code> (autograd)
Optimizer	manual $W \leftarrow W - lr * dW$	<code>optimizer="sgd"</code>	<code>torch.optim.SGD(...)</code>
Training loop	explicit for epoch / for batch	<code>model.fit(...)</code>	explicit for epoch / for batch
Evaluation	manual metric functions	<code>model.evaluate</code> -equivalent (manual)	manual (<code>model.eval()</code> , <code>no_grad</code>)
Prediction	<code>forward(X_test, params)</code>	<code>model.predict(...)</code>	<code>model(x_test)</code> under <code>torch.no_grad()</code>

5.7 Training Curves & Test Results

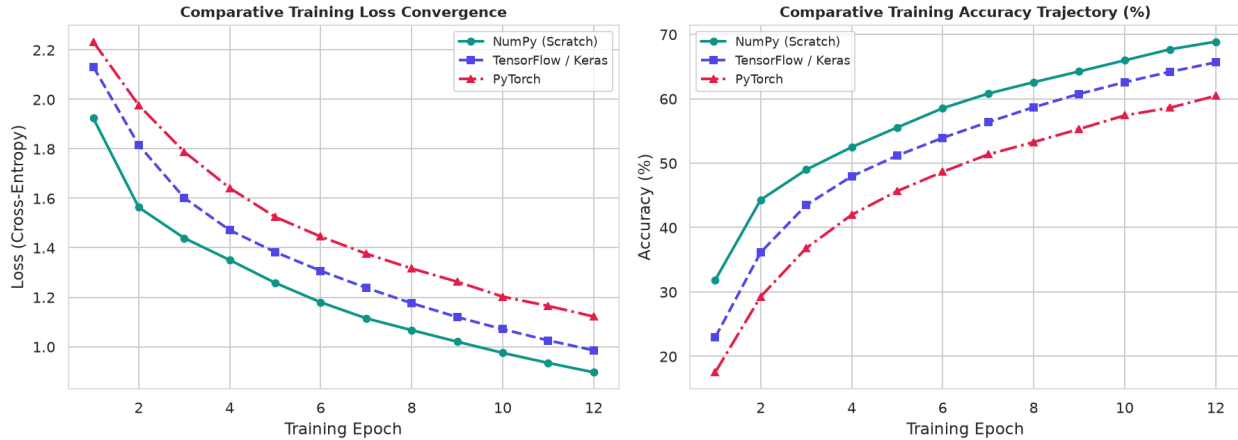


Figure 21: Cross-implementation empirical loss descent profiles across 12 training epochs on CIFAR-10.

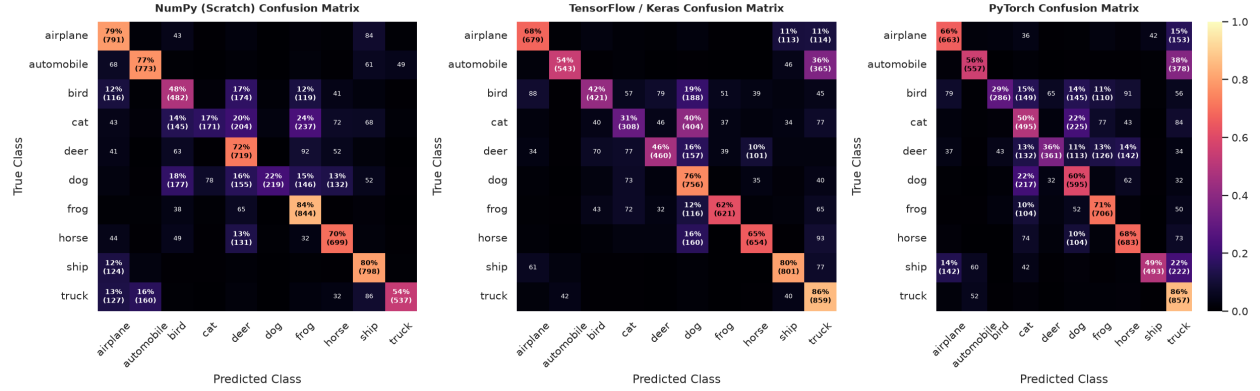


Figure 22: Evaluation confusion matrices across implementations on the CIFAR-10 test set ($N = 10,000$); misclassification density concentrates along natural visual boundaries between cats and dogs, and between cars and trucks.

Table 8: CIFAR-10: quantitative comparison, test set

Implementation	Params	Train time (s)	Final loss	Accuracy	Precision (macro)	Recall (macro)
Scratch	545,098	1,875.8	0.8965	0.6033	0.6335	0.6033
Keras	545,098	103.3	0.9854	0.6102	0.6477	0.6102
PyTorch	545,098	244.1	1.1220	0.5696	0.6163	0.5696

Macro F1 scores evaluate to: 0.5794 (Scratch), 0.6078 (Keras), and 0.5655 (PyTorch).

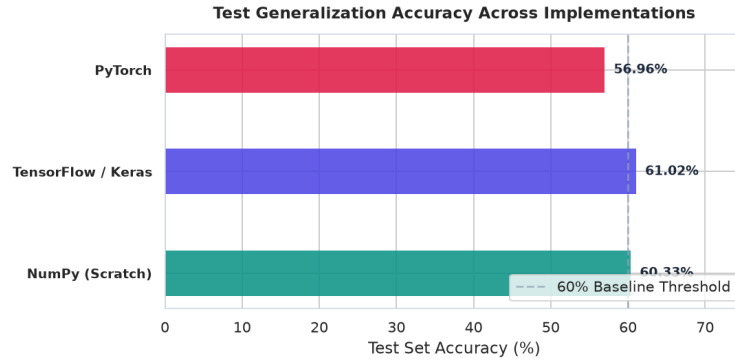


Figure 23: Comparative generalization accuracy on CIFAR-10; moderate scores in the 57%–61% range reflect the heightened complexity of natural photographic recognition without data augmentation.

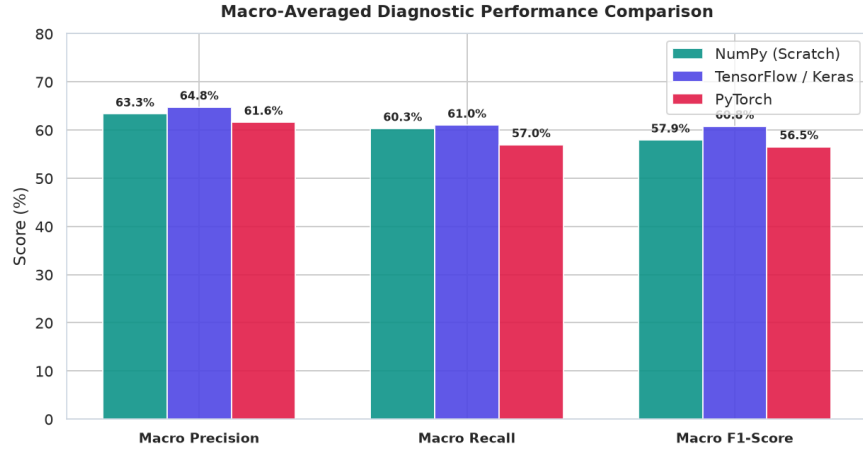


Figure 24: Macro-averaged precision, recall, and F1 scores across the three software implementations on CIFAR-10.



Figure 25: Trainable parameter footprint (545,098 weights) alongside wall-clock training duration, illustrating the widest execution throughput divergence observed across all benchmarks.

5.8 Runtime Speed & Comparison

Keras achieves top accuracy (0.610), followed closely by Scratch (0.603) and PyTorch (0.570). All models converge within a four-point window. Accuracy in the low 60s reflects the difficulty of 10-class photographic classification on 32×32 images without augmentation, rather than implementation shortcomings.

Training time highlights the widening performance gap: Scratch required 1,875.8 seconds (≈ 31 minutes), whereas Keras took 103.3 seconds and PyTorch took 244.1 seconds. Scratch was **18.2x slower** than Keras and **7.7x slower** than PyTorch. Expanding input spatial size and channel depth ($32 \times 32 \times 3$ vs. $28 \times 28 \times 1$) multiplies `im2col` patch extraction volume, a cost pure NumPy absorbs directly in CPU cycles while compiled framework backends optimize efficiently.

6 Overall Cross-Task Comparison

Table 9 synthesizes test accuracy across all nine experimental conditions (3 tasks $\times$ 3 implementations).

Table 9: Cross-application summary: test accuracy, all 3 applications x 3 implementations

Application	Implementation	Parameters	Accuracy
Diabetes	Scratch	2,177	0.7310
	Keras	2,177	0.7533
	PyTorch	2,177	0.7819
Fashion-MNIST	Scratch	421,642	0.8973
	Keras	421,642	0.8603
	PyTorch	421,642	0.8660
CIFAR-10	Scratch	545,098	0.6033
	Keras	545,098	0.6102
	PyTorch	545,098	0.5696

Bold typography denotes peak validation accuracy within each benchmark. Trainable parameter footprints are strictly invariant across all three software implementations within each task, verified via automated assertions in every computational notebook.

Table 10: Cross-application training time, seconds, all 3 applications x 3 implementations

Application	Scratch	Keras	PyTorch
Diabetes	69.4	24.7	311.0
Fashion-MNIST	1,330.5	75.6	191.5
CIFAR-10	1,875.8	103.3	244.1

Bold typography highlights the most computationally efficient implementation per task. TensorFlow/Keras achieves the highest computational throughput across all three domains. The pure NumPy baseline is competitive with PyTorch only on tabular MLP data, while lagging dramatically on both convolutional image tasks.

No individual implementation wins every task. Furthermore, the highest-accuracy model on Diabetes (PyTorch) is also the slowest, demonstrating that predictive accuracy and training throughput represent independent axes of framework evaluation.

7 Discussion & Lessons Learned

7.1 Did the Same Architecture Truly Hold?

Yes. Parameter footprints matched exactly across all three implementations in every application, confirmed by runtime assertions. In addition, accuracy spreads remained bounded within four to five percentage points across all tasks. The rotation of top-performing frameworks (PyTorch on Diabetes, Scratch on Fashion-

MNIST, Keras on CIFAR-10) directly supports Lecture 04’s core premise: when the mathematical architecture is held invariant, no framework possesses inherently superior statistical learning capacity.

7.2 Where Abstraction Levels Genuinely Differed

Training runtime is where software abstractions diverged sharply. On the tabular Diabetes MLP, Scratch and Keras trained in comparable times (69.4s vs. 24.7s), while PyTorch was slowest (311.0s) due to fixed per-batch autograd graph construction overhead that a small 2,177-parameter model cannot amortize. On convolutional networks, this pattern reversed completely: Scratch was 17.6x slower than Keras on Fashion-MNIST and 18.2x slower on CIFAR-10. This widening gap demonstrates that `im2col` patch extraction in interpreted Python scales poorly with input spatial resolution and channel depth, whereas compiled framework backends leverage optimized C++/CUDA SIMD routines.

7.3 Practical Insights from Building by Hand

Developing implementations from scratch uncovered two critical implementation details that high-level APIs silently mask:

- **Loss Formulation Divergence:** PyTorch’s `BCEWithLogitsLoss(pos_weight=...)` scales only the positive error term, shifting the scalar loss baseline upwards (0.87 vs. 0.51) compared to Scratch and Keras, which scale both classes symmetrically. This demonstrates that identical functional concepts can be parameterized differently across libraries.
- **Gradient Check Verification:** Finite-difference gradient checking on the Fashion-MNIST scratch model caught a redundant $1/N$ division in the dense layer backward pass that had artificially damped gradients while still producing plausible loss descent. Numerical checking proved indispensable for ensuring gradient fidelity.

8 Conclusion & Summary

This empirical study validates the central pedagogical premise of Lecture 04: neural architectures represent invariant mathematical functions whose statistical capacity is independent of software abstraction tiers. Across all three benchmarks, parameter counts matched identically and generalization accuracies converged within tight bounds. However, computational runtime diverged dramatically: on convolutional tasks, compiled framework engines delivered up to 18.2x speedups over pure NumPy. Handcrafting training loops and loss functions exposed important implementation nuances—such as asymmetric loss weighting in PyTorch and subtle backward gradient scaling bugs—demonstrating the unique pedagogical value of implementing deep learning algorithms from foundational principles.

9 Reproducibility Guide

9.1 Hardware & Environment Setup

All benchmarks were executed in an isolated virtual environment at `assignment_04/.venv` (Python 3.10.11, kernel `assignment_04`) using: NumPy, Pandas, Scikit-Learn, Matplotlib, Seaborn, Jupyter, NbConvert, TensorFlow 2.21.0, PyTorch 2.14.0+cpu, and Torchvision 0.29.0+cpu. CPU-only packages ensured fair runtime comparisons without hardware confounds. All random number generators were locked to `RANDOM_SEED = 42`.

9.2 How to Re-run the Notebooks

Notebooks can be re-executed autonomously from their respective directories:

```
../.venv/Scripts/python.exe -m jupyter nbconvert \  
--to notebook --execute --inplace \  
notebook/<app>.ipynb --ExecutePreprocessor.timeout=1800
```

substituting <app> with `diabetes`, `fashion_mnist`, or `cifar10`. Datasets for Fashion-MNIST and CIFAR-10 download automatically via `keras.datasets`, whereas Diabetes requires CSV files in `diabetes/data/`. Upon execution, notebooks serialize trained weights into `model/`:

- `scratch_weights.npz`: NumPy scratch trained weights.
- `keras_model.keras`: exported Keras model.
- `pytorch_model.pt`: PyTorch state dictionary.
- `feature_names.joblib` (Diabetes only): encoded tabular feature column names.
- `input_schema.json`: documented input attributes and target schema.

A validation cell in each notebook reloads all artifacts to verify prediction consistency.